

Que 1 →

- Create 1 Public Docker Hub registry named flask_webapp_yourname.
- Clone below repository on your system.

<https://github.com/mmumshad/simple-webapp-docker.git>

- Initialize a local repository & copy the code from above repo to your local repository in

your working branch.

- Once code is copied to the local repository, build the docker image & add meaningful

tags with version 1 and push to Docker Hub registry.

- Once Images are pushed to Docker hub registries, create a directory named kube.

Inside the kube directory create deployment.yaml file with 3 replication , labels app:

flask-webapp , containerPort: 8080 and add the image that you have pushed in Docker

Hub registry.

- Create one service.yaml file with type nodeport & select flask-webapp with port 8080 &

targetPort 8080 with any nodePort between range 30000-32768.

- Once a service is created try accessing the web page in the browser as below. (30011 is

nodeport mentioned in service.yaml). Meanwhile open app.py from your code to understand paths & output.

`http://master_ip:30011/`

`http://master_ip:30011/how are you`

- Now , update the app.py from your code and add below route above if `__name__ ==`

`"__main__"` line

```
@app.route('/Who are you')
```

```
def cloudeithix():
```

```
    return 'Yes, I am cloudeithix, and You !!!'
```

- Once the file is updated , rebuild the docker image & add meaningful tags with version 2

and push to Docker Hub registry.

- Now we have the latest docker image in repo, It's time to roll out a new image. Roll out

the new Image with all three ways one by one.

1. With kubectl set command
2. With kubectl edit deployment
3. With deployment.yaml file modification.

- Run the # kubectl rollout command to check status and history.

- Note:- Once above step 1 is done , run # kubectl rollout undo deployment command to

rollback the change and then try a second way of rollout.

- In the browser run all three routes & notice the changes.

`http://master_ip:30011/`

`http://master_ip:30011/how are you`

`http://master_ip:30011/Who are you`

- Once done with all above steps , commit all the changes to the remote repository.

- Capture the snap and prepare a well formatted document.

achal@LAPTOP-# git clone <https://github.com/mmumshad/simple-webapp-docker.git>

Cloning into 'simple-webapp-docker'...

remote: Enumerating objects: 14, done.

remote: Counting objects: 100% (7/7), done.

```
remote: Compressing objects: 100% (5/5), done.
remote: Total 14 (delta 3), reused 2 (delta 2), pack-reused 7 (from 1)
Receiving objects: 100% (14/14), done.
Resolving deltas: 100% (3/3), done.

-----

achal@LAPTOP-# cp -r simple-webapp-docker/ local-repo/
achal@LAPTOP-HPUOTJF1:Assignment-practice-3$ cd local-repo/
achal@LAPTOP-HPUOTJF1:local-repo$ ls
simple-webapp-docker

-----

achal@LAPTOP-# podman build -t docker.io/achalpadol/flask_webapp_achal:v1 .
STEP 1/5: FROM ubuntu:20.04
Resolved "ubuntu" as an alias
(/etc/containers/registries.conf.d/shortnames.conf)
Trying to pull docker.io/library/ubuntu:20.04...
Getting image source signatures
Copying blob 13b7e930469f done    |
Copying config b7bab04fd9 done    |
Writing manifest to image destination
STEP 2/5: RUN apt-get update && apt-get install -y python3 python3-pip
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:3 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [1300 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [4710 kB]
Get:5 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [4368 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 Packages [32.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages [1275 kB]

-----

achal@LAPTOP-# podman images
REPOSITORY                                     TAG          IMAGE
ID          CREATED          SIZE
docker.io/achalpadol/flask_webapp_achal        v1
949f9230f443  23 seconds ago  480 MB

-----

achal@LAPTOP-# docker push docker.io/achalpadol/flask_webapp_achal:v1
Getting image source signatures
Copying blob 5a8ac36787a8 done    |
Copying blob 8168e02bd347 done    |
Copying blob 721125e8a881 done    |
Copying blob 13b7e930469f skipped: already exists
Copying config 949f9230f4 done    |
Writing manifest to image destination

-----

achal@LAPTOP-# cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-webapp-1
  labels:
    app: flask-webapp-1
spec:
  replicas: 3
  selector:
    matchLabels:
```

```
    app: flask-webapp-1
  template:
    metadata:
      labels:
        app: flask-webapp-1
    spec:
      containers:
        - name: flask-webapp-1
          image: docker.io/achalpadol/flask_webapp_achal:v1
          ports:
            - containerPort: 8080
```

```
achal@LAPTOP-#:kube$ kubectl get deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
flask-webapp-1	3/3	3	3	2m29s

```
achal@LAPTOP-#:kube$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
flask-webapp-1-74458c66d7-6dt2p	1/1	Running	0	2m42s
flask-webapp-1-74458c66d7-hk4nd	1/1	Running	0	2m42s
flask-webapp-1-74458c66d7-knwd8	1/1	Running	0	2m42s

```
achal@LAPTOP-# cat service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: flask-webapp-service
spec:
  type: NodePort
  selector:
    app: flask-webapp-1
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30011
```

```
achal@LAPTOP-#:kube$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
flask-webapp-service	NodePort	10.100.2.192	<none>	8080:30011/TCP

```
achal@LAPTOP-#:kube$ curl http://192.168.49.2:30011
```

Welcome!

```
achal@LAPTOP-#:kube$ curl "http://192.168.49.2:30011/how%20are%20you"
```

I am good, how about you?

```
achal@LAPTOP-# cat app.py
```

```
import os
from flask import Flask
app = Flask(__name__)
```

```
@app.route("/")
def main():
    return "Welcome!"
```

```
@app.route('/how are you')
def hello():
    return 'I am good, how about you?'
```

```
if __name__ == "__main__":
    app.run()

-----

# updated app.py

achal@LAPTOP-# cat app.py
import os
from flask import Flask
app = Flask(__name__)

@app.route("/")
def main():
    return "Welcome!"

@app.route('/how are you')
def hello():
    return 'I am good, how about you?'

@app.route('/Who are you')
def cloudethix():
    return 'Yes, I am cloudethix, and You !!!'

if __name__ == "__main__":
    app.run()

-----

achal@LAPTOP-# podman build -t docker.io/achalpadol/flask_webapp_achal:v2 .

STEP 1/5: FROM ubuntu:20.04
STEP 2/5: RUN apt-get update && apt-get install -y python3 python3-pip
--> Using cache
eb374c6995cc5c9da545c53818fb15b63b9d3ea3cda6309be11299a130876107
--> eb374c6995cc
STEP 3/5: RUN pip install flask
--> Using cache
f8d2a7f56585a18b5799133c76d7a2f3ed3afe3d4f84a013e73997307bfafd19
--> f8d2a7f56585
STEP 4/5: COPY app.py /opt/
--> ab831a5f60a7
STEP 5/5: ENTRYPOINT FLASK_APP=/opt/app.py flask run --host=0.0.0.0 --
port=8080
COMMIT docker.io/achalpadol/flask_webapp_achal:v2
--> 43ecf906970a
Successfully tagged docker.io/achalpadol/flask_webapp_achal:v2
43ecf906970a397be9d2c4c65836b34b26b8dde28c170c6c15059ee1e5068ebc

-----

achal@LAPTOP-# podman images

REPOSITORY                                     TAG      IMAGE
ID          CREATED          SIZE
docker.io/achalpadol/flask_webapp_achal      v2
43ecf906970a  4 seconds ago  480 MB

-----

achal@LAPTOP-# podman images | grep flask_webapp
docker.io/achalpadol/flask_webapp_achal      v2
43ecf906970a  About a minute ago  480 MB
docker.io/achalpadol/flask_webapp_achal      v1
949f9230f443  14 hours ago      480 MB

-----

achal@LAPTOP-# podman push docker.io/achalpadol/flask_webapp_achal:v2
```

```
Getting image source signatures
Copying blob 6ac90cd9bb76 done    |
Copying blob 13b7e930469f skipped: already exists
Copying blob d59147826128 skipped: already exists
Copying blob bde202e56e37 skipped: already exists
Copying config 43ecf90697 done    |
Writing manifest to image destination

-----

achal@LAPTOP-# kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
flask-webapp-1      3/3     3            3           135m

-----

achal@LAPTOP-# kubectl describe deployment flask-webapp-1 | grep Image
Image:              docker.io/achalpadol/flask_webapp_achal:v1

-----

achal@LAPTOP-# kubectl set image deployment/flask-webapp-1 flask-webapp-1=docker.io/achalpadol/flask_webapp_achal:v2
deployment.apps/flask-webapp-1 image updated

-----

achal@LAPTOP-# kubectl describe deployment flask-webapp-1 | grep Image
Image:              docker.io/achalpadol/flask_webapp_achal:v2

-----

achal@LAPTOP-# kubectl rollout status deployment/flask-webapp-1
deployment "flask-webapp-1" successfully rolled out

-----

achal@LAPTOP-# kubectl rollout history deployment/flask-webapp-1
deployment.apps/flask-webapp-1
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

-----

achal@LAPTOP-# kubectl rollout undo deployment/flask-webapp-1
Warning: resource deployments/flask-webapp-1 was previously managed with
'kubectl apply'. Rolling back will not update the kubectl.kubernetes.io/last-
applied-configuration annotation, which may cause unexpected behavior on
future 'kubectl apply' operations. Consider using 'kubectl apply' with your
previous configuration file instead.
deployment.apps/flask-webapp-1 rolled back

-----

achal@LAPTOP-# kubectl describe deployment flask-webapp-1 | grep Image
Image:              docker.io/achalpadol/flask_webapp_achal:v1

-----

# Edit deployment

achal@LAPTOP-# kubectl edit deployment flask-webapp-1
deployment.apps/flask-webapp-1 edited

achal@LAPTOP-# kubectl rollout status deployment/flask-webapp-1
deployment "flask-webapp-1" successfully rolled out
achal@LAPTOP-# kubectl rollout history deployment/flask-webapp-1
deployment.apps/flask-webapp-1
REVISION  CHANGE-CAUSE
3         <none>
4         <none>

-----

# undo again to check third step

achal@LAPTOP-# kubectl rollout undo deployment/flask-webapp-1
Warning: resource deployments/flask-webapp-1 was previously managed with
'kubectl apply'. Rolling back will not update the kubectl.kubernetes.io/last-
```

applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous configuration file instead.

deployment.apps/flask-webapp-1 rolled back

```
achal@LAPTOP-# kubectl describe deployment flask-webapp-1 | grep Image
Image:          docker.io/achalpadol/flask_webapp_achal:v1
```

```
achal@LAPTOP-# kubectl rollout status deployment/flask-webapp-1
deployment "flask-webapp-1" successfully rolled out
```

```
achal@LAPTOP-# kubectl rollout history deployment/flask-webapp-1
deployment.apps/flask-webapp-1
REVISION  CHANGE-CAUSE
4         <none>
5         <none>
```

#with deployment.yaml file modification.

```
achal@LAPTOP-# cat deployment.yaml
```

```
achal@LAPTOP-# cat deployment.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: flask-webapp-1
```

```
  labels:
```

```
    app: flask-webapp-1
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: flask-webapp-1
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: flask-webapp-1
```

```
    spec:
```

```
      containers:
```

```
        - name: flask-webapp-1
```

```
          image: docker.io/achalpadol/flask_webapp_achal:v2
```

```
          ports:
```

```
            - containerPort: 8080
```

```
achal@LAPTOP-# kubectl apply -f deployment.yaml
```

```
deployment.apps/flask-webapp-1 configured
```

```
achal@LAPTOP-# kubectl rollout status deployment/flask-webapp-1
deployment "flask-webapp-1" successfully rolled out
```

```
achal@LAPTOP-# kubectl rollout history deployment/flask-webapp-1
deployment.apps/flask-webapp-1
REVISION  CHANGE-CAUSE
5         <none>
6         <none>
```

```
achal@LAPTOP-# curl http://192.168.49.2:30011
Welcome!
```

```
achal@LAPTOP-# curl "http://192.168.49.2:30011/how%20are%20you"
I am good, how about you?
```

```
achal@LAPTOP-# curl http://192.168.49.2:30011/Who%20are%20you
Yes, I am cloudethix, and You !!!
```

```
-----
achal@LAPTOP-# git branch -M main
achal@LAPTOP-HPUOTJF1:local-repo$ git remote add origin
https://github.com/achalpadol/cloudethix-rollout-repo-2.git
```

```
achal@LAPTOP-# git remote -v
origin https://github.com/achalpadol/cloudethix-rollout-repo-2.git (fetch)
origin https://github.com/achalpadol/cloudethix-rollout-repo-2.git (push)
```

```
achal@LAPTOP-# git push -u origin main
Username for 'https://github.com': achalpadol
Password for 'https://achalpadol@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 670 bytes | 670.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/achalpadol/cloudethix-rollout-repo-2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
-----
Que 2 →
```

- Download and install Lens & access your k8s cluster from Lens.
- Create nginx Pod and Nodeport service. Check the Pod logs from Lens.
- Check the service from lens. Also login to the pod shell using the lens.
- Take snaps and delete the resources that you have just created.

```
-----
achal@LAPTOP-# minikube dashboard
```

```
🔧 Enabling dashboard ...
  ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
  ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all
features please run:
```

```
minikube addons enable metrics-server
```

```
🤖 Verifying dashboard health ...
🚀 Launching proxy ...
🤖 Verifying proxy health ...
🎉 Opening http://127.0.0.1:33983/api/v1/namespaces/kubernetes-
dashboard/services/http:kubernetes-dashboard:/proxy/ in your default
browser...
👉 http://127.0.0.1:33983/api/v1/namespaces/kubernetes-
dashboard/services/http:kubernetes-dashboard:/proxy/
```

```
-----
achal@LAPTOP-# cat pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-nginx
  labels:
    app: app-nginx
spec:
  containers:
    - name: app-nginx
      image: nginx:latest
```

```
ports:
  - containerPort: 80
```

```
achal@LAPTOP-# cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: NodePort-Service
spec:
  selector:
    app: app-nginx
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Que 3 →

- Create 1 Public Docker Hub registry named `cloudethix_configmap_yourname`.
- Clone below repository on your system.

<https://github.com/zembutsu/docker-sample-nginx.git>

- Initialize a local repository & copy the code from above repo to your local repository in the working branch.

- Once code is copied , build a docker image from docker file and add meaningful tags

and push to docker hub repository.

- Once Images are pushed to Docker hub registries, create a directory named kube.

Inside the kube directory create `deployment.yaml` file with 3 replication , labels app:

`frontend-webapp` , `containerPort: 80` and add the image that you have pushed in Docker

Hub registry.

- Create one `service.yaml` file with type `nodeport` & select `frontend-webapp` pod with port

`80` & `targetPort 80` with any `nodePort` between range `30000-32768`.

- Once the service is created try accessing the web page in the browser as below. Notice

the changes & take the snap.

- Now create a `configmap.yaml` file with below data & delete the deployment that you have created.

```
<html>
```

```
<body>
```

```
<h1> I am Cloudethix Team, Are you ?!! </h1>
```

```
Version: 1.1
```

```
</body>
```

```
</html>
```

- Then update the same `deployment.yaml` file and mount `configmap` as volume on container using `volumeMounts` with `mountPath /usr/share/nginx/html/`

- Now it's time to create `configmap` & `deployment`. Once created , try to access the

webpage in the browser & confirm that the index page is the same as we have in `configmap`.

```
achal@LAPTOP-# git clone https://github.com/zembutsu/docker-sample-nginx.git
Cloning into 'docker-sample-nginx'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (12/12), done.
```



```
remote: Compressing objects: 100% (6/6), done.
remote: Total 22 (delta 7), reused 6 (delta 6), pack-reused 10 (from 1)
Receiving objects: 100% (22/22), done.
Resolving deltas: 100% (7/7), done.
```

```
achal@LAPTOP-# podman build -t
docker.io/achalpadol/cloudethix_configmap_achal:v1 .
```

```
WARN[0000] "/" is not a shared mount, this could cause issues or missing
mounts with rootless containers
STEP 1/3: FROM nginx:alpine
STEP 2/3: COPY default.conf /etc/nginx/conf.d/
--> Using cache
090b497e1161956bc69c29ed05f63003fabe8615f06a8b8a09cc43f4e2783826
--> 090b497e1161
STEP 3/3: COPY index.html /usr/share/nginx/html/
--> Using cache
97ebac17772705f64c7fb37106bd497011c968ff7fd48c4a0f6d8233a32ea5b
COMMIT docker.io/achalpadol/cloudethix_configmap_achal:v1
--> 97ebac177727
Successfully tagged docker.io/achalpadol/cloudethix_configmap_achal:v1
Successfully tagged docker.io/achalpadol/cloudethix-nginx-achal:v1
Successfully tagged localhost/achalpadol/cloudethix-nginx-achal:v1
Successfully tagged localhost/cloudethix-nginx-achal:v1
97ebac17772705f64c7fb37106bd497011c968ff7fd48c4a0f6d8233a32ea5b
```

```
achal@LAPTOP-# podman images | grep cloudethix_configmap
docker.io/achalpadol/cloudethix_configmap_achal      v1
97ebac177727    24 hours ago    64.2 MB
```

```
achal@LAPTOP-# podman push docker.io/achalpadol/cloudethix_configmap_achal:v1
Getting image source signatures
Copying blob 0d62d88506ba skipped: already exists
Copying blob d94291c26261 skipped: already exists
Copying blob 1ed8e39a7434 skipped: already exists
Copying blob 8cdfef4f23778 skipped: already exists
Copying blob 0ea935727878 skipped: already exists
Copying blob 55afa1ecc21d skipped: already exists
Copying blob fb597529c916 skipped: already exists
Copying blob da4dea1b00af skipped: already exists
Copying blob 0cc7f3a48f3f skipped: already exists
Copying blob 4b794759b385 skipped: already exists
Copying config 97ebac1777 done    |
Writing manifest to image destination
```

```
achal@LAPTOP-# cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-webapp-1
  namespace: cloudethix-achal
  labels:
    app: frontend-webapp-1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend-webapp-1
  template:
    metadata:
```

```
      labels:
        app: frontend-webapp-1
    spec:
      containers:
        - name: frontend-webapp-1
          image: docker.io/achalpadol/cloudethix_configmap_achal:v1
          ports:
            - containerPort: 80
```

```
achal@LAPTOP-# cat service.yml
apiVersion: v1
kind: Service
metadata:
  name: frontend-webapp-service
  namespace: cloudethix-achal
spec:
  type: NodePort
  selector:
    app: frontend-webapp-1
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30012
```

```
achal@LAPTOP-# minikube service frontend-webapp-service --url -n cloudethix-achal
http://127.0.0.1:40131
! Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

```
^Cachal@LAPTOP-# kubectl get pod
```

NAME	READY	STATUS	RESTARTS
AGE			
frontend-webapp-1-74bc7cdf58-7bdbm	1/1	Running	0
5m25s			
frontend-webapp-1-74bc7cdf58-rnrvq	1/1	Running	0
5m25s			
frontend-webapp-1-74bc7cdf58-vxxmr	1/1	Running	0
5m25s			

```
achal@LAPTOP-# kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
frontend-webapp-service	NodePort	10.110.136.109	<none>	80:30012/TCP
5m5s				

```
achal@LAPTOP-# curl http://127.0.0.1:40131
<!DOCTYPE html>
<html>
<head>
  <title>CloudEthiX Nginx</title>
</head>
<body>
  <h1>CloudEthiX</h1>
  <h2>Hostname: nginx</h2>
</body>
</html>
```

```
achal@LAPTOP-# cat configmap.yml
apiVersion: v1
```

```
kind: ConfigMap
metadata:
  name: frontend-html
  namespace: cloudethix-achal
data:
  index.html: |
    <html>
    <body>
    <h1> I am Cloudethix Team, Are you ?!! </h1>
    Version: 1.1
    </body>
    </html>
```

```
-----
achal@LAPTOP-HPUOTJF1:kube$ kubectl apply -f configmap.yml
configmap/frontend-html created
```

```
^Cachal@LAPTOP-HPUOTJF1:kube$ kubectl get cm
NAME          DATA  AGE
frontend-html  1      3m41s
```

```
-----
# updated deployment.yml
achal@LAPTOP-# cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-webapp-1
  namespace: cloudethix-achal
  labels:
    app: frontend-webapp-1

spec:
  replicas: 3

  selector:
    matchLabels:
      app: frontend-webapp-1

  template:
    metadata:
      labels:
        app: frontend-webapp-1

    spec:
      containers:
        - name: frontend-webapp-1
          image: docker.io/achalpadol/cloudethix_configmap_achal:v1
          ports:
            - containerPort: 80

          volumeMounts:
            - name: nginx-html
              mountPath: /usr/share/nginx/html/

          volumes:
            - name: nginx-html
              configMap:
                name: frontend-html
```

```
-----
achal@LAPTOP-# kubectl get deployment
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
```

frontend-webapp-1 3/3 3 3 7m54s

```
achal@LAPTOP-# minikube service frontend-webapp-service --url -n cloudehix-achal
http://127.0.0.1:33741
! Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

```
achal@LAPTOP-HPUOTJF1:~$ curl http://127.0.0.1:33741
<html>
<body>
<h1> I am Cloudehix Team, Are you ?!! </h1>
Version: 1.1
</body>
</html>
```

Que 4 →

- Create 1 Public Docker Hub registry named `cloudehix_multicontainer_yourname`.
- Clone below repository on your system.
`https://github.com/janakiramm/Kubernetes-multi-container-pod.git`
- Initialize a local repository & copy the code from above repo to your local repository in any of your working branches.
- Once code is copied , go to the Build directory and build docker image from docker file and add meaningful tags and push to docker hub repository.
- Now go to the deploy directory and notice the files.
- Here, `web-pod-1.yml` file will create the pod with two containers (Multi container). Take a note of labels , name of containers and ports. Also, please make sure you will update the python container image that you have pushed to your docker registry.
- Now, open `web-svc.yml` file and notice service Type , selectors & targetPort. Apply the file.
- Now open `db-pod.yml` & notice the labels , name , Image, containerPort and apply the file.
- Now open the `db-svc.yml` file and notice service Type , selectors & targetPort. Apply the file.
- Once above files are applied , Check that the Pods and Services are created using command line or lens.
- Now , from the command line run below urls & notice the changes.
`curl http://$NODE_IP:$NODE_PORT/init`
Initialize the database with sample schema
- Now it's time to Insert some sample data. Make sure you will use correct `$NODE_IP:$NODE_PORT`
`curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "1", "user": "John Doe"}' http://$NODE_IP:$NODE_PORT/users/add`
`curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "2", "user": "Jane Doe"}' http://$NODE_IP:$NODE_PORT/users/add`
`curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "3", "user": "Bill Colls"}' http://$NODE_IP:$NODE_PORT/users/add`
`curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "4", "user": "Mike Taylor"}' http://$NODE_IP:$NODE_PORT/users/add`
- Now access the data that we have added to database using below command.
`curl http://$NODE_IP:$NODE_PORT/users/1`

- The second time you access the data, it appends '(c)' indicating that it is pulled from the Redis cache.
- Also, try to access mysql shell i.e db pod & run select * from the users table. check app.py for DB related information.
- Prepare proper documentation in brief & write start to end flow. Refer below link if you face any issues.
<https://github.com/janakiramm/Kubernetes-multi-container-pod>

```
-----
achal@LAPTOP-# git clone https://github.com/janakiramm/Kubernetes-multi-
container-pod.git
Cloning into 'Kubernetes-multi-container-pod'...
remote: Enumerating objects: 51, done.
remote: Total 51 (delta 0), reused 0 (delta 0), pack-reused 51 (from 1)
Receiving objects: 100% (51/51), 88.14 KiB | 247.00 KiB/s, done.
Resolving deltas: 100% (21/21), done.
achal@LAPTOP-HPUOTJF1:task-4$ ls
Kubernetes-multi-container-pod
-----
```

```
achal@LAPTOP-# podman build -t
docker.io/achalpadol/cloudethix_multicontainer_achal:v1 .
STEP 1/3: FROM python:2.7-onbuild
Resolved "python" as an alias
(/etc/containers/registries.conf.d/shortnames.conf)
Trying to pull docker.io/library/python:2.7-onbuild...
Getting image source signatures
Copying blob 0c3de61682aa done      |
Copying blob 46dde23c37b3 done      |
Copying blob d660b1f15b9b done      |
Copying blob e7428f935583 done      |
Copying blob 56f10ddf1173 done      |
Copying blob 6ebaeb074589 done      |
Copying blob 4473537c621d done      |
Copying blob 3106f7df3d1c done      |
Copying blob 3de1c6ceef68 done      |
Copying config 3f246dd60a done      |
Writing manifest to image destination
STEP 2/6: COPY requirements.txt /usr/src/app/
--> 88f14b23e568
STEP 3/6: RUN pip install --no-cache-dir -r requirements.txt
Collecting flask (from -r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packa
-----
```

```
achal@LAPTOP-# podman push
docker.io/achalpadol/cloudethix_multicontainer_achal:v1
Getting image source signatures
Copying blob 6ebaeb074589 skipped: already exists
Copying blob 0c3de61682aa skipped: already exists
Copying blob 46dde23c37b3 skipped: already exists
Copying blob d660b1f15b9b skipped: already exists
Copying blob 56f10ddf1173 skipped: already exists
Copying blob e7428f935583 skipped: already exists
Copying blob 4f0e84c5ce6c done      |
Copying blob 4cf08394a795 done      |
Copying blob 0ba33796ade2 done      |
Copying blob 3de1c6ceef68 skipped: already exists
Copying blob 3106f7df3d1c skipped: already exists
Copying blob 4473537c621d skipped: already exists
```

Copying config 7997bb4680 done |
Writing manifest to image destination

```
achal@LAPTOP-# cat web-pod-1.yml
apiVersion: "v1"
kind: Pod
metadata:
  name: web1
  labels:
    name: web
    app: demo
spec:
  containers:
    - name: redis
      image: redis
      ports:
        - containerPort: 6379
          name: redis
          protocol: TCP
    - name: python
      image: docker.io/achalpadol/cloudethix_multicontainer_achal:v1
      env:
        - name: "REDIS_HOST"
          value: "localhost"
      ports:
        - containerPort: 5000
          name: http
          protocol: TCP
```

```
achal@LAPTOP-# cat web-svc.yml
apiVersion: v1
kind: Service
metadata:
  name: web
  labels:
    name: web
    app: demo
spec:
  selector:
    name: web
  type: NodePort
  ports:
    - port: 80
      name: http
      targetPort: 5000
      protocol: TCP
```

```
achal@LAPTOP-#:Deploy$ cat db-pod.yml
apiVersion: "v1"
kind: Pod
metadata:
  name: mysql
  labels:
    name: mysql
    app: demo
spec:
  containers:
    - name: mysql
      image: mysql:5.7.25
      ports:
```

```
      - containerPort: 3306
        protocol: TCP
    env:
      -
        name: "MYSQL_ROOT_PASSWORD"
        value: "password"
```

```
achal@LAPTOP-#:Deploy$ cat db-svc.yml
apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    name: mysql
    app: demo
spec:
  ports:
    - port: 3306
      name: mysql
      targetPort: 3306
  selector:
    name: mysql
    app: demo
```

```
achal@LAPTOP-# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
mysql         1/1     Running   0           10m
web1          2/2     Running   0           50s
```

```
achal@LAPTOP-# kubectl get svc
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
mysql     ClusterIP   10.101.187.146  <none>       3306/TCP         5m47s
web       NodePort    10.98.202.95    <none>       80:31992/TCP     7m59s
```

```
achal@LAPTOP-# curl http://localhost:8080/init
DB Init doneachal@LAPTOP-HPUOTJF1:~$
```

```
achal@LAPTOP-# curl http://localhost:8080/init
DB Init doneachal@LAPTOP-curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "1",{"uid": "1",
"user":"John Doe"}' http://localhost:8080/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 09:57:59 GMT
```

```
Addedachal@LAPTOP-# curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "2", "2",
"user":"Jane Doe"}' http://localhost:8080/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 09:58:46 GMT
```

```
Addedachal@LAPTOP-# curl -i -H "Content-Type: application/json" -X POST -d '{"uid": "3", "3",
"user":"Bill Colls"}' http://localhost:8080/users/add
HTTP/1.0 200 OK
```

```
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 09:59:24 GMT
-----
Addedachal@LAPTOP-# curl -i -H "Content-Type: application/json" -X POST -d
'{"uid": "4", "4",
"user":"Mike Taylor"}' http://localhost:8080/users/add
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 5
Server: Werkzeug/1.0.1 Python/2.7.15
Date: Thu, 03 Sep 2026 09:59:50 GMT
-----
Addedachal@LAPTOP-# curl http://localhost:8080/users/1ers/1
John Doeachal@LAPTOP-HPU0TJF1:~$

chal@LAPTOP-# kubectl exec -it mysql -n cloudethix-achal -- mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5
Server version: 5.7.25 MySQL Community Server (GPL)

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases:
-> ;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use near
':.' at line 1
mysql> SHOW DATABASES;
+-----+
| Database          |
+-----+
| information_schema |
| USERDB             |
| mysql              |
| performance_schema |
| sys                |
+-----+
5 rows in set (0.01 sec)

mysql> use USERDB;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_USERDB |
+-----+
| users             |
+-----+
1 row in set (0.00 sec)
```



```
mysql> select * from users;
+-----+-----+
| ID    | USER          |
+-----+-----+
| 1     | John Doe      |
| 2     | Jane Doe      |
| 3     | Bill Colls    |
| 4     | Mike Taylor   |
+-----+-----+
4 rows in set (0.00 sec)
```

-
- Que 5 →
- Create 1 Public Docker Hub registry named `cloudethix_Initcontainer_yourname`.
 - Clone below repository on your system.
`https://github.com/janakiramm/simpleapp.git`
 - Initialize a local repository & copy the code from above repo to your local repository in any of your working branch.
 - Once code is copied , go to the Build directory and build docker image from docker file and add meaningful tags and push to docker hub repository.
 - Once Images are pushed to Docker hub registries, create a directory named `kube`. Inside the kube directory create `deployment.yaml` file with 3 replication , label `app`: `simpleapp-webapp` , `containerPort: 80` and add the image that you have pushed in Docker Hub registry.
 - Create one `service.yaml` file with type `nodeport` & select `simpleapp-webapp` pod with port `80` & `targetPort 80` with any `nodePort` between range `30000-32768`.
 - Open the webpage in the browser and notice the changes and capture the snap.
 - Then delete the deployment that you have just created.
 - Update the `deployment.yaml` file and add `volumeMounts` with `mountPath /usr/share/nginx/html` from `emptyDir: {}` volume.
 - Once above changes are added, add `initContainers` block with below parameters. Also add `volumeMounts` for Init Container with `mountPath "/work-dir"` from `emptyDir: {}` volume.
- ```
initContainers:
- name: install
image: busybox:1.28
command:
- wget
- "-O"
- "/work-dir/index.html"
- http://info.cern.ch
volumeMounts:
- name: workdir
mountPath: "/work-dir"
```
- Add volumes with `emptyDir: {}` in `deployment.yaml` file.
  - Once the `deployment.yaml` file is ready, create the deployment & access the page in the browser and notice the changes.
  - Prepare a well formatted document and write your understanding step by step.

-----

```
achal@LAPTOP-# git clone https://github.com/janakiramm/simpleapp.git
Cloning into 'simpleapp'...
```

```
remote: Enumerating objects: 47, done.
remote: Total 47 (delta 0), reused 0 (delta 0), pack-reused 47 (from 1)
Receiving objects: 100% (47/47), 8.20 KiB | 1.64 MiB/s, done.
Resolving deltas: 100% (9/9), done.

achal@LAPTOP-#:local-repo$ ls
Dockerfile html wrapper.sh
achal@LAPTOP-HPU0TJF1:local-repo$ git init
hint: Using 'master' as the name for the initial branch. This default branch
name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this
warning,
hint: call:
hint:
hint: git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint: git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName
false"
Initialized empty Git repository in /home/achal/Assignment-practice-3/task-
5/local-repo/.git/

achal@LAPTOP-#$ podman images
REPOSITORY TAG IMAGE
ID CREATED SIZE
docker.io/achalpadol/cloudethix_multicontainer_achal v1
7687bb45809e 2 hours ago 712 MB

achal@LAPTOP-# podman push
docker.io/achalpadol/cloudethix_initcontainer_achal:v1
Getting image source signatures
Copying blob 5310eb16bf43 skipped: already exists
Copying blob a44b5c8be516 skipped: already exists
Copying blob c13f294dea34 skipped: already exists

achal@LAPTOP-# cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: simpleapp-webapp-1
spec:
 replicas: 3
 selector:
 matchLabels:
 app: simpleapp-webapp-1
 template:
 metadata:
 labels:
 app: simpleapp-webapp-1
 spec:
 containers:
 - name: simpleapp-webapp-1
 image: docker.io/achalpadol/cloudethix_initcontainer_achal:v1
 ports:
 - containerPort: 80
```

```

achal@LAPTOP-# k get deployment
NAME READY UP-T0-DATE AVAILABLE AGE
simpleapp-webapp-1 3/3 3 3 3m35s

achal@LAPTOP-#k get pod
NAME READY STATUS RESTARTS AGE
simpleapp-webapp-1-6c56c976fc-5mpfj 1/1 Running 0 3m52s
simpleapp-webapp-1-6c56c976fc-cw5mh 1/1 Running 0 3m52s
simpleapp-webapp-1-6c56c976fc-xrmcv 1/1 Running 0 3m52s

achal@LAPTOP-# cat service.yml
apiVersion: v1
kind: Service
metadata:
 name: simpleapp-service-1
spec:
 type: NodePort
 selector:
 app: simpleapp-webapp-1
 ports:
 - port: 80
 nodePort: 30009

achal@LAPTOP-# k get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
AGE
simpleapp-service-1 NodePort 10.98.49.99 <none> 80:30009/TCP
89s

achal@LAPTOP-# curl http://127.0.0.1:44855
<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8">
 <title>Sample Deployment</title>
 <style>
 body {
 color: #ffffff;
 background-color: blue;
 font-family: Arial, sans-serif;
 font-size: 14px;
 }

 h1 {
 font-size: 500%;
 font-weight: normal;
 margin-bottom: 0;
 }

 h2 {
 font-size: 200%;
 font-weight: normal;
 margin-bottom: 0;
 }
 </style>
</head>
<body>
 <div align="center">
 <h1>Welcome to V1 of the web application</h1>
```

```
<h2>This application will be deployed on Kubernetes.</h2>
</div>
</body>
</html>
```

---

```
achal@LAPTOP-# k delete deployment simpleapp-webapp-1
deployment.apps "simpleapp-webapp-1" deleted from cloudethix-achal namespace
achal@LAPTOP-HPUOTJF1:kube$ k get deployment
No resources found in cloudethix-achal namespace.
```

---

```
Update deployment.yml file
achal@LAPTOP-# cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: simpleapp-webapp
 namespace: cloudethix-achal
```

```
spec:
 replicas: 3
```

```
 selector:
 matchLabels:
 app: simpleapp-webapp-1
```

```
 template:
 metadata:
 labels:
 app: simpleapp-webapp-1
```

```
 spec:
```

```
 initContainers:
 - name: install
 image: busybox:1.28
 command:
 - wget
 - "-O"
 - "/work-dir/index.html"
 - "http://info.cern.ch"
 volumeMounts:
 - name: workdir
 mountPath: "/work-dir"
```

```
 containers:
 - name: simpleapp-webapp-1
 image: docker.io/achalpadol/cloudethix_initcontainer_achal:v1
 ports:
 - containerPort: 80
```

```
 volumeMounts:
 - name: workdir
 mountPath: /usr/share/nginx/html
```

```
 volumes:
 - name: workdir
 emptyDir: {}
```

---

```
achal@LAPTOP-# k apply -f deployment.yml
deployment.apps/simpleapp-webapp-1 created
```

```
achal@LAPTOP-# k get deployment
NAME READY UP-TO-DATE AVAILABLE AGE
simpleapp-webapp-1 3/3 3 3 10s
achal@LAPTOP-# k get pod

NAME READY STATUS RESTARTS AGE
simpleapp-webapp-1-5dc7b6d554-4cjh 1/1 Running 0 14s
simpleapp-webapp-1-5dc7b6d554-qlzzq 1/1 Running 0 14s
simpleapp-webapp-1-5dc7b6d554-xfkpx 1/1 Running 0 14s

achal@LAPTOP-# curl http://127.0.0.1:46571
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>

Browse the
first website
<a href="http://line-
mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using
the line-mode browser simulator
Learn about the birth
of the web
Learn about CERN, the physics
laboratory where the web was born

</body></html>

Que 6 →
• Create 1 Public Docker Hub registry named cloudethix_hpa_yourname.
• Clone below repository on your system.
https://github.com/vivekamin/kubernetes-hpa-example.git
• Initialize a local repository & copy the code from above repo to your local
repository in
any of your working branch.
• Once code is copied , build a docker image from the docker file and add
meaningful tags
and push to the docker hub repository.
• Once the image is pushed, go to k8s directory and update deployment.yaml
file with
image name from your repo. And then create it.
• Open service.yml and change the type to nodePort and apply the same.
• Open the HPA.yaml file, notice it and then apply the same.
• Open the browser, and access the webpage.
• Now it's time to test the HPA working with the below command.
kubectl run -i --tty load-generator --rm --image=busybox --restart=Never --
/bin/sh -c "while sleep 0.01; do wget -q -O- http://NODE_PORT_SERVICE_NAME;
done"
• Check the HPA from kubectl command and also check if the deployment is
scaling up.
• Take the snap , prepare a well formatted doc and write your understanding.

achal@LAPTOP-# git clone https://github.com/vivekamin/kubernetes-hpa-
example.git
Cloning into 'kubernetes-hpa-example'...
remote: Enumerating objects: 26, done.
remote: Total 26 (delta 0), reused 0 (delta 0), pack-reused 26 (from 1)
```

Receiving objects: 100% (26/26), done.

Resolving deltas: 100% (9/9), done.

---

achal@LAPTOP-# git init

hint: Using 'master' as the name for the initial branch. This default branch name

hint: will change to "main" in Git 3.0. To configure the initial branch name

hint: to use in all of your new repositories, which will suppress this warning,

hint: call:

hint:

hint: git config --global init.defaultBranch <name>

hint:

hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and

hint: 'development'. The just-created branch can be renamed via this command:

hint:

hint: git branch -m <name>

hint:

hint: Disable this message with "git config set advice.defaultBranchName false"

Initialized empty Git repository in /home/achal/Assignment-practice-3/task-6/local-repo/.git/

---

achal@LAPTOP-# sudo podman build -t

docker.io/achalpadol/cloudethix\_hpa\_achal:v1 .

STEP 1/6: FROM node:8.12.0-alpine

achal@LAPTOP-HPUOTJF1:cloudethix\_hpa\_achal\$ sudo podman images

REPOSITORY	TAG	IMAGE ID
CREATED	SIZE	
docker.io/achalpadol/cloudethix_hpa_achal	v1	f83e042dbcd5 45
seconds ago	69.2 MB	

---

achal@LAPTOP-# cat deployment.yml

apiVersion: apps/v1

kind: Deployment

metadata:

name: node-example

spec:

replicas: 1

selector:

matchLabels:

app: node-example

template:

metadata:

labels:

app: node-example

spec:

containers:

- name: node-example

image: docker.io/achalpadol/cloudethix\_hpa\_achal:v1

imagePullPolicy: Always

ports:

- containerPort: 3000

resources:

limits:

cpu: "0.5"

requests:

cpu: "0.25"

```

achal@LAPTOP-# k apply -f deployment.yml
deployment.apps/node-example created
achal@LAPTOP-HPUOTJF1:local-repo$ k get pod
NAME READY STATUS RESTARTS AGE
node-example-68f464997b-29p9h 0/1 Pending 0 8s

```

```
achal@LAPTOP-# cat service.yml
apiVersion: v1
kind: Service
metadata:
 name: node-example
 labels:
 app: node-example
spec:
 selector:
 app: node-example
 ports:
 - port: 3000
 protocol: TCP
 nodePort: 30001
 type: NodePort

```

```
achal@LAPTOP-# k apply -f service.yml
service/node-example created
achal@LAPTOP-HPUOTJF1:local-repo$ k get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
AGE
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP
3d16h
node-example NodePort 10.103.61.92 <none> 3000:30001/TCP
4s

```

```
achal@LAPTOP-# k apply -f hpa.yml
horizontalpodautoscaler.autoscaling/node-example configured
achal@LAPTOP-HPUOTJF1:k8s$ k get hpa
NAME REFERENCE TARGETS MINPODS MAXPODS
REPLICAS AGE
node-example Deployment/node-example cpu: 0%/1% 1 4 1
3d

```

```
achal@LAPTOP-# minikube service node-example --url
http://127.0.0.1:45331
! Because you are using a Docker driver on linux, the terminal needs to be
open to run it.

achal@LAPTOP-# curl http://127.0.0.1:45331
Hello World!!

```

```
^Cachal@LAPTOP-HPUOTJF1:k8s$ k top pod
NAME CPU(cores) MEMORY(bytes)
node-example-68f464997b-29p9h 1m 27Mi

```

- Que 7 →
- Create 1 Public Docker Hub registry named cloudethix\_cronjob\_yourname.
  - Initialize a local repository & copy below code (three files) to your local repository in any of your working branch.
  - Once code is copied, build the docker image from Dockerfile , add meaningful

tags and  
then push the docker image to Docker hub registry.

- Now update the pythoncronjob.yml file to change the image name that you have just pushed to docker hub registry.
- Now create a cron job using pythoncronjob.yml file. Check with kubectl command if the cron job is created.
- Check the Job name which is created by cronjob from command line or lens.
- Then check the pod logs which are created by the job and capture the output.
- Prepare well formatted documents and write your understanding.

```

achal@LAPTOP-# cat Dockerfile
FROM python:3.7-alpine
#add user group and ass user to that group
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
#creates work dir
WORKDIR /app
#copy python script to the container folder app
COPY helloworld.py /app/helloworld.py
RUN chmod +x /app/helloworld.py
#user is appuser
USER appuser
ENTRYPOINT ["python", "/app/helloworld.py"]

```

```
achal@LAPTOP-# docker build -t cloudehix_cronjob_achal:v4.
STEP 1/7: FROM python:3.7-alpine
Resolved "python" as an alias
(/etc/containers/registries.conf.d/shortnames.conf)
Trying to pull docker.io/library/python:3.7-alpine...
Getting image source signatures

```

```
chal@LAPTOP-# podman tag cloudehix_cronjob_achal:v4
docker.io/achalpadol/cloudehix_cronjob_achal:v4
achal@LAPTOP-HPUOTJF1:cloudehix_cronjob_achal$ podman images
REPOSITORY TAG IMAGE
ID CREATED SIZE
localhost/cloudehix_cronjob_achal v4
0e0635316d06 2 minutes ago 49.5 MB
docker.io/achalpadol/cloudehix_cronjob_achal v4
0e0635316d06 2 minutes ago 49.5 MB

```

```
achal@LAPTOP-# podman push docker.io/achalpadol/cloudehix_cronjob_achal:v4
Getting image source signatures
Copying blob c43d3c0ede05 done |
Copying blob 148762f75a1f skipped: already exists
Copying blob 4819c95424fc skipped: already exists
Copying blob 9875af95546d skipped: already exists
Copying blob ea1518237b37 skipped: already exists

```

```
achal@LAPTOP-# k apply -f pythoncronjob.yml
cronjob.batch/python-helloworld created
achal@LAPTOP-HPUOTJF1:task-7$ k get pod
NAME READY STATUS RESTARTS AGE
python-helloworld-29812643-8cfhn 0/1 Completed 0 2m11s
python-helloworld-29812644-grh59 0/1 Completed 0 71s
python-helloworld-29812645-xhzbj 0/1 Completed 0 11s

```



